



Topic :

Assignment - 04

Objective :

Outcomes :

Q.1 Explain the hierarchy of Computer Memory systems with Examples

OR

What is memory Organization? Discuss different types of Memory Organization in Computer systems?

The hierarchy of Computer memory systems is organized into multiple levels. Each with different characteristics capacities and access speeds. These levels are arranged in a hierarchy based on their proximity to the CPU and their speeds.

1. Registers

- Registers are the fastest and smallest type of memory directly accessible by the CPU.
- They store data and instructions that are currently being processed by the CPU
- Examples including GPRs, PC, IR

2. Cache Memory

- Cache memory is small but faster type of a memory located between the CPU and main memory.
- It stores copies of frequently accessed data and instructions from main memory to speed up CPU access



→ Examples Level 1 (L1), Level 2 (L2) and Level 3 (L3) caches. Each with varying capacities and access speeds.

3. Main Memory (RAM)

- It is also known as (RAM), is the primary memory used by the CPU to store data and instructions during program execution.
- It is larger and slower than Cache memory but faster than secondary storage devices.
- Examples. DRAM, SRAM used in computer system for temporary data storage.

4. Secondary Storage

- provides non-volatile, long term storage for data and programs.
- They have larger capacities but slower access speeds compared to main memory.
- Examples HDD, SSD, CD, DVD and magnetic tapes.

5. Tertiary Storage

- Refers to archival storage systems used for long term data retention and backup.
- These devices offer even larger capacities but slower access speeds compared to secondary storage.
- Examples. tape libraries, and cloud storage services.



Q2. Explain Paging. Explain how paging can be implemented in the CPU to access virtual memory?

Paging is a memory management scheme used in computer operating systems to implement virtual memory. In paging, physical memory is divided into fixed size blocks called pages, and logical memory (virtual address space) is divided into blocks of the same size called page frames. Paging allows the operating system to map virtual addresses to physical addresses efficiently, enabling processes to use more memory than physically available and providing memory protection and isolation.

Paging can be implemented in the CPU to access virtual memory by.

Address Translation:

When a program accesses memory, it generates a virtual address which contains a page number and an offset within the page. The CPU's memory management unit (MMU) translates the virtual address into a physical address using a page table, which is maintained by the operating system.



The page table maps virtual page numbers to physical page frame numbers, allowing the CPU to determine the physical location of the data being accessed.

Page Table

It is a data structure used by the MMU to perform address translation.

It consists of Page Table Entries (PTE) which contain information about a single page in virtual memory, such as the corresponding page frame number in physical memory and various control bits.

The page table is typically stored in physical memory and managed by the operating system. When a program accesses memory, the MMU uses the page table to look up the physical address corresponding to the virtual address.

Page Table Walk

To translate a virtual address into a physical address, the MMU performs a page table walk which involves multiple memory accesses.

The MMU uses the virtual page number to index into the page table's top level entry, known as the page directory or page table directory (PTD).



The PTD contains Pointers to Page tables, which are used to translate the Virtual page number further. The MMU follows these Pointers through multiple levels of page tables until it reaches the final page table Entry containing the physical Page frame number.

Page Fault Handling

If a Virtual page is not present in physical memory (i.e., a page fault occurs), the operating system is responsible for handling the page fault.

The Operating System selects a Page to Evict from physical memory (if necessary) and brings the required Page into memory from disk, updating the page table accordingly. Once the page is in memory, the CPU can continue the memory access operation as usual.

Translation Lookaside Buffer (TLB)

To speed up address translation, many CPUs include a Translation Lookaside Buffer (TLB) which is a small, fast cache for page table Entries.

The TLB stores recently accessed page table Entries, allowing the MMU to perform address



Sushila Devi Bansal College of Technology

A.B. Road, Near Rau, Umaria, Indore

Q6

translation without accessing the Page table in memory.

If a Virtual address is found in the TLB (a TLB hit), the corresponding physical address can be obtained quickly.

If the Virtual address is not found in the TLB (a TLB miss), the MMU access the page table in memory to retrieve the required Page table Entry.

Paging allows the OS to provide virtual memory support to process efficiently.

Q3.



Q3. What is Cache memory? Name three techniques of Cache mapping and Explain any one in detail. Write Steps to improve Cache performance?

Cache memory is a small, high speed memory unit located between the CPU (Central processing unit) and main memory (RAM) in a computer system. Its purpose is to store frequently accessed data and instructions to reduce the average time required to access memory. Cache memory operates on the principle of locality, exploiting the fact that programs tend to access the same memory locations repeatedly or access nearby memory locations in a short period.

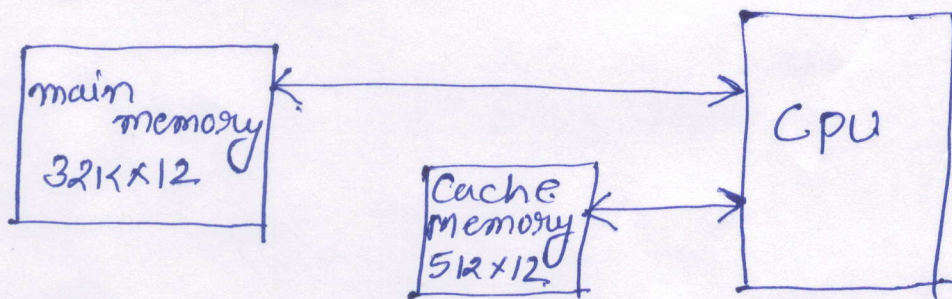


Fig. Example of cache memory

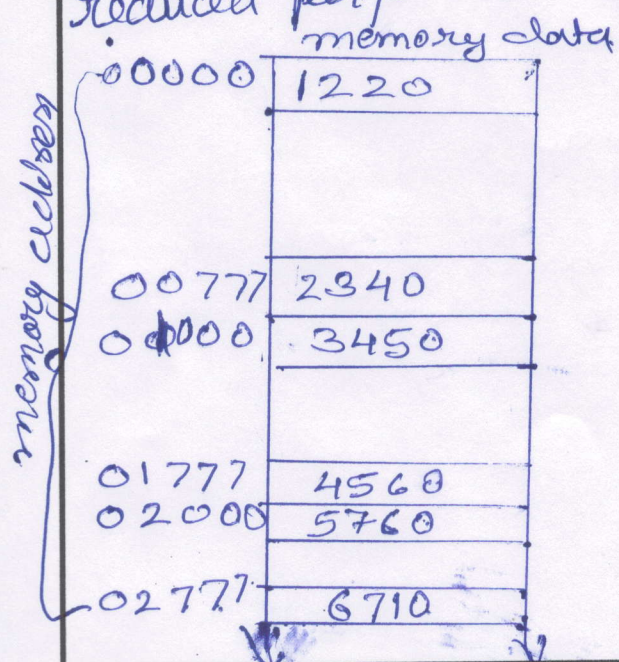
Cache mapping techniques determine how data from main memory is mapped to cache memory. There are three main techniques of cache mapping are.



- 1) Direct mapping.
- 2) Associative mapping
- 3) ~~Full~~ Set Associative Mapping

1) Direct mapping

- Each block of main memory can be mapped to only one specific block in the cache.
- Mapping is determined by using a hash function or modulus operation on the memory address to select the cache block.
- Direct mapping is simple to implement and requires less hardware overhead.
- However, it may lead to higher cache conflicts (where multiple memory blocks map to the same cache block), resulting in cache thrashing and reduced performance.



Index Address	tag	Data
000	00	1220
777	02	6710

(b) cache memory

(a) main memory

Fig Direct mapping cache organization



Improving Cache performance involves optimizing cache utilization, reducing cache misses, and minimizing cache access latency.

- 1) Increase Cache Size
- 2) Use Higher Associativity
- 3) Optimize Cache Replacement Policy
- 4) Prefetching
- 5) Cache Blocking
- 6) Compiler Optimization
- 7) Hardware Prefetching
- 8) Cache Coherence
- 9) Reduce Cache Access Latency



Q. Consider a 2-way set associative mapped cache of size 16 KB with block size 256 bytes. The size of main memory is 128 KB. Find the number of bits in the tag directory size.

To find the number of bits in the tag directory size of for a 2-way set associative cache we need to calculate the following

Total number of set in cache
number of bits required to represent each set in the cache.
number of bits required to represent the tag for each block.

We have

$$\text{Cache Size} = 16 \text{ KB} = 16 \times 1024 \text{ bytes}$$

$$\text{Block Size} = 256 \text{ bytes}$$

$$\begin{aligned} \text{No. of Sets in the cache} \\ = \text{Cache Size} / (\text{Set Size} \times \text{Block Size}) \end{aligned}$$

$$\begin{aligned} \text{Set Size} &= \text{No. of lines in the cache} \\ &= \text{Cache Size} / (\text{Associativity} \times \text{Block Size}) \end{aligned}$$

$$\text{main memory size} = 128 \text{ KB} = 128 \times 1024 \text{ bytes}$$

Let's calculate.

No. of Sets in the Cache:

$$\text{Cache Size} = 16 \times 1024 \text{ bytes}$$

$$\text{Block Size} = 256 \text{ bytes}$$

$$\text{Associativity} = 2 \text{ (2-way set associative)}$$



$$\begin{aligned}\text{No. of Sets} &= \text{cache Size} / (\text{Associativity} \times \text{Block Size}) \\ &= (16 \times 1024) / (2 \times 256) \\ &= 32 \text{ Sets}\end{aligned}$$

No. of bits required to represent Each Set in the cache

$$\begin{aligned}\rightarrow \boxed{\text{No of bits}} &= \log_2 (\text{No. of Sets}) \\ &= \log_2 (32) \\ &= 5 \text{ bits}\end{aligned}$$

No. of bits required to represent the tag for Each block

$$\begin{aligned}\boxed{\text{Total no of bits in the address}} &= \log_2 (\text{main memory size}) \\ &= \log_2 (128 \times 1024) \\ &= 17 \text{ bits}\end{aligned}$$

$$\begin{aligned}\text{No. of index bits (Set bits)} &= \log_2 (\text{No. of Set}) \\ &= 5 \text{ bits}\end{aligned}$$

$$\begin{aligned}\boxed{\text{No. of offset bits (block bits)}} &= \log_2 (\text{Block size}) \\ &= \log_2 (256) \\ &= 8 \text{ bits}\end{aligned}$$

$$\begin{aligned}\text{No. of tag bits} &= (\text{Total no of bits in the address}) - (\text{no of index bits}) - (\text{No. of offset bits}) \\ &= 17 - 5 - 8 \\ &= 4 \text{ bits}\end{aligned}$$



for a 2-way set associative cache set requires two tags (One for Each Cache line)

∴ The tag directory size is the product of the number of sets and the number of bits required to represent tag for each block.

$$\begin{aligned}\text{Tag directory size} &= (\text{No. of sets}) \times (\text{No. of tags per set}) \\ &\quad \times (\text{No. of bits per tag}) \\ &= 32 \text{ sets} \times 2 \text{ tags per set} \times 4 \text{ bits per tag} \\ &= 256 \text{ bits}\end{aligned}$$

So, the number of bits in the tag directory size is 256 bits



Q5 Describe the page replacement algorithms and Explain any three page replacement methods?

Page replacement algorithms are used in operating systems with virtual memory to decide which page to Evict from memory (i.e., swap out) when a new page needs to be loaded into memory (i.e., Swapping) and the memory is full.

The goal of page replacement algorithm is to minimize the number of page faults and optimize system performance by selecting the most appropriate page to Evict based on certain criteria.

Three page replacement algorithms:

Optimal page Replacement (OPT or MIN)

The optimal page replacement algorithm is an ~~optional~~ optimal but impractical algorithm used for benchmarking other page replacement algorithms. OPT replaces the page that will not be used for the longest period in the future, as determined by examining the future memory access ~~times~~ patterns.



Since predicting future memory access is not feasible in practice, OPT is often used as reference for comparing the performance of other page replacement algorithms.

While OPT provides the lowest possible number of page faults (optimal performance), it requires knowledge of future memory access patterns, which is typically not available.

FIFO (First-in-First Out)

It replaces the oldest page in memory, i.e., the page that has been in memory for the longest time.

FIFO is implemented using a queue data structure where pages are loaded into memory in sequential order and evicted in the same order.

While FIFO is simple to implement and has low overhead, it suffers from the "Belady's anomaly" where increasing the number of frames can lead to an increase in the number of page faults (contrary to the intuition that more memory should reduce page faults).



Least Recently Used (LRU)

The LRU page replacement algorithm replaces the page that has not been accessed for the longest period.

LRU keeps track of the order in which pages are accessed and selects the page that was least recently used for eviction.

Implementing an accurate LRU algorithm requires maintaining a data structure (e.g., a queue or a list) to track the access time of each page.

While LRU provides better performance than FIFO and is not susceptible to the Belady's anomaly, its implementation complexity and overhead can be significant, especially in systems with large memory sizes.



Compare following.

(1) Main Memory and Auxiliary Memory

Comparison between main memory and Auxiliary memory on the basis of following

Definition

main memory, also known as primary memory or RAM, is a volatile memory component directly accessible to the CPU. It holds data and instructions that are actively being used by the CPU for processing.

Auxiliary Memory, also known as secondary memory or storage, refers to non-volatile storage devices used for long term data storage.

Volatility

main memory is volatile.

Auxiliary memory is non-volatile.

Access Time:

Main Memory offers fast access time.

Auxiliary Memory devices have slower access time compared to main memory.

Capacity

main memory is more expensive per unit of storage compared to Auxiliary Memory.



Auxiliary memory devices are typically less expensive per unit of storage. Compare to main memory.

(ii) Cache Memory and Virtual Memory

Cache memory and virtual memory are both important components of a computer's memory hierarchy, but they serve different purposes and operate at different levels of the memory hierarchy.

Purpose:

Cache memory is used to store frequently accessed data and instructions that are currently being processed by the CPU.

The primary purpose of Cache Memory is to reduce the average access time to frequently used data and instructions by providing faster access compared to main memory.

Virtual Memory is a memory management technique that allows the operating system to provide the illusion of having more physical memory (RAM) than is actually available.

VM enables efficient utilization of limited physical memory by transparently swapping data between main memory and auxiliary storage as needed.



Location

Cache memory is located between the CPU and main memory (RAM) in the memory hierarchy.

Cache memory typically implemented as a small, high speed memory unit directly accessible by the CPU.

Virtual memory is a logical memory abstraction managed by the operating system.

Virtual memory resides on disk storage and is managed by the operating system's memory management unit (MMU) and memory management algorithms.

Size and Capacity

Cache memory is relatively small in size compared to main memory (RAM) and auxiliary storage.

Its capacity depends on factors such as cost, technology, limitations, and physical space constraints.

Virtual Memory ~~access~~ can be much larger in size compared to physical main memory.

Size of virtual memory is limited by the available disk space and the system's address space limitations.

Access Time:

Cache memory access time is fast.

Virtual memory access time is slow.



(3) Paging and Segmentation

Both are memory management techniques used in OS to organize and manage the memory space while both techniques aim to provide a logical view of memory to processes. They differ in their approach and implementation. Let's compare paging and segmentation.

Basic Concept

Paging In paging physical memory is divided into fixed blocks called pages, and logical memory is divided into fixed-size blocks called frames. Paging eliminates the need for contiguous allocation of physical memory.

Segmentation In this the logical address space is

divided into variable-sized segments. Each segment represents a logical unit of program (such as code, data, stack).

It allows the programmer to view memory as a collection of segments, each with its own attributes and permissions.



Memory Allocation.

~~process~~ paging - processes are allocated memory in contiguous blocks of pages

Segmentation - processes are allocated memory in non contiguous segments

Address Translation

paging Address translation in paging involves mapping logical addresses (page numbers) to physical addresses (frame numbers) using page tables. Each page table entry contains the mapping information for a page.

Segmentation Address translation in this involves mapping logical addresses (segment numbers) to physical addresses using segment tables. Each segment table entry contains the base address and size of a segment.

Fragmentation

Paging provides memory protection at the page level. It can lead to an internal fragmentation where the last page of a process may not be fully utilized, resulting in wasted memory within a page.



Segmentation It can lead to External fragmentation where free memory is fragmented into small holes that cannot be allocated to processes efficiently.

(4) Hit and Miss Ratio.

They are performance metrics used to evaluate the effectiveness of cache memory systems.

Hit Ratio (Hit Rate)

It measures the percentage of memory accesses the result in cache hits.

It is calculated by

$$\text{Hit Ratio} = \left(\frac{\text{No. of cache hits}}{\text{Total no. of memory accesses}} \right) \times 100\%$$

It indicates that a significant portion of memory accesses are satisfied by the cache, resulting in improved system performance.

Miss Ratio (Miss Rate)

measures percentage of memory accesses that result in cache misses.

Calculated by.

$$\text{Miss Ratio} = \left(\frac{\text{No. of cache misses}}{\text{Total no. of memory accesses}} \right) \times 100\%$$

It represents that a significant portion of memory accesses cannot be satisfied by the cache, resulting in increased access latency and potentially reduced system performance.



Q7 Write Short notes on the following.

(i) Semiconductor memories

Semiconductor memories are a type of digital data storage device that use semiconductor technology to store and retrieve data electronically. These memories are widely used in various electronic devices, including computers, smart-phones, digital cameras and more.

These memories offer several advantages over traditional magnetic or optical storage technologies including faster access time, lower power consumption, smaller physical size and greater shock resistance.

E.g. RAM

ROM

Flash Memory

Cache Memory.

(2) Memory Management Hardware.

Refers to the components and circuits within a computer system responsible for controlling and organizing the flow of data between the CPU, main memory and peripheral devices.



These hardware components work together to manage the allocation, access, and retrieval of data in the system's memory hierarchy. Some of the key memory management hardware components include.

Memory management Unit (MMU)

Responsible for translating virtual addresses generated by the CPU into physical addresses in main memory.

It uses paging and segmentation for addresses translation.

Memory Controller

Responsible for managing the flow of data between the CPU and main memory.

Cache Controller

Responsible for managing the operation of the CPU's cache memory hierarchy.

Memory Interfacing circuits

Responsible for communication between the CPU and external memory devices.

DMA Controller

It is a specialized hardware component that enables high speed transmission between peripheral devices.



and main memory without CPU intervention.

(3) Translation Lookaside Buffer (TLB)

The TLB is a hardware cache used in modern Computer Systems to improve the performance of Virtual memory management.

It is small, high speed cache memory that stores recently used Virtual-to-physical address translations allowing for faster address translation and reducing the overhead associated with accessing main memory.

TLB works for

Address Translation

Cache for Address translation

Reduced Access times

Cache Management

(4) Average Access time

Average amount of time taken to access data from a memory or storage device, taking into account all possible access scenarios and their latencies.

It is important metrics for evaluating the performance of memory systems and is influenced



by factors such as access latency, Cache hit/miss ratios, and memory hierarchy organization.

~~To calculate~~^{for} Average Access time depends on the Specific memory system Architecture and hierarchy of memory levels involved

Average access time provides a measure of the overall Efficiency and performance of the memory system taking into account both the speed and hierarchy of different memory levels.